[CS3704] Intermediate Software Design and Engineering

Dr. Chris Brown
Virginia Tech
10/13/2026

Announcements

- HW2 due tomorrow (10/14) before midnight
  - Requirements and Design Activities
- PM2.1 due (Development Sprint I) due 10/23 before midnight
- Ut Prosim opportunities
- Feedback survey
  - Please complete by end of this week.
- Dr. Brown office hour change
  - Wednesday (11-12) -> Thursday (11-12)

Project 2 Questions

- How substantial do the changes to the original project have to be?/How complex should each of the three required enhancements be? **[Team decision]**
- Are there any requirements for tests coverage for our unit tests? **[TBD, team will set own requirements]**
- Can all three enhancements we complete be ideas from the original team? **[Yes]**
- Can enhancements build upon each other in each sprint? Or do we need three different ideas before we start the sprints? **[Team decision]**
- What do you mean by Pipeline in PM2.3, is this referring to the pipes in Github? **[CI/CD pipeline (11/3 lecture)]**

Project 2 Questions (cont.)

- Are there any restrictions on what types of enhancements we can make to the existing application? **[Not really]**
- Are we expected to maintain all existing functionality while adding new features? **[Yes]**
- How will individual contributions be evaluated within the group? **[Contributions to milestones, GitHub repository, team-reported]**

Implementation and Maintenance

Implementation Overview
Code Quality and Analysis
Maintenance
Project 2 Install-Fest

Learning Outcomes

By the end of the course, students should be able to:

- **Understand software engineering processes, methods, and tools used in the software development life cycle (SDLC)**
- Use techniques and processes to create and analyze requirements for an application
- Use techniques and processes to design a software system
- **Identify processes, methods, and tools related to phases of the SDLC**
- Explain the differences between software engineering processes
- Discuss research questions and current topics related to software engineering
- Create and communicate about the requirements and design of a software application

Warm-Up (5 minutes)



1. Open https://pydle.net/
2. Complete the coding challenge individually or in pairs.
   - Requires basic Python knowledge
3. Discuss the following:
   a. How did you determine your approach was good or bad?
   b. If applicable, what evidence helped you revise your approach?
   c. What would make the code easier to maintain or improve?
   d. Is number of characters a good or bad coding metric?

Implementation

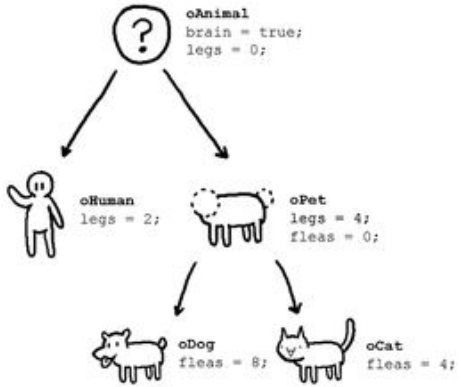


Goal: translate software design into a concrete system (i.e. write code)

- Can use any programming language, but some languages are better suited to certain types of programs than others
- SE/implementation is a *team* activity!
- *Software Artifacts*: source code, documentation, configuration files, media, executables, bug database, source code repository, issue trackers…

Design to Implementation

What OOP users claim



Code Quality

Software metrics provide a systematic way to characterize the quality of software based on a set of clearly defined rules.

Why study code quality?

- Assists in the evaluation and analysis of *design*;
- Provides an indication of the complexity of *code*;
- Enhances development and *maintenance*;
  - Predicting errors, optimizing programs, enforce consistent coding styles, evaluating team and process productivity,...
- Facilitates more effective *testing*.

Code Metrics

- Lines of Code
- Cyclomatic Complexity
- Halstead Complexity
- Weighted Methods per Class (*object-oriented*)
- Inheritance Tree Depth (*object-oriented*)
- Number of Children (*object-oriented*)

Maintainability Index (MI): is a composite metric that incorporates a number of traditional source code metrics into a single number that indicates relative **maintainability**.

Lines of Code

The number of lines of code in a program.

- More lines of code = more maintenance

- **LOC is difficult to measure.**
  - How do we actually count lines of code?
    - i.e. comments, multi-line statements, blanks,…

Example: How many lines of code is this?

```
1  def hello():
2      print("Hello CS5704")
3
4  if __name__ == "__main__":
5      hello()
6
```

`wc -l`

5

`sloc test.py`

```
---------- Result ------------
            Physical :  5
              Source :  4
             Comment :  0
 Single-line comment :  0
       Block comment :  0
               Mixed :  0
 Empty block comment :  0
               Empty :  1
               To Do :  0

Number of files read :  1

------------------------------
```

`cloc test.py`

```
github.com/AlDanial/cloc v 1.88  T=0.01 s (183.1 files/s, 915.7 lines/s)
-------------------------------------------------------------------------------
Language                     files          blank        comment           code
-------------------------------------------------------------------------------
Python                           1              1              0              4
-------------------------------------------------------------------------------
```

Cyclomatic Complexity

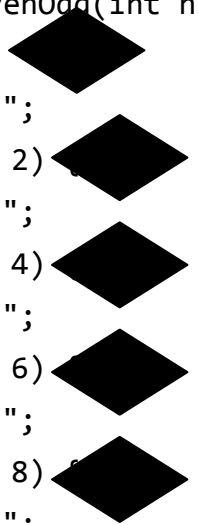
The number of independent paths in a program.

- Measures the number of linear segments in a method (branches)
- Useful for measuring the *minimum* number of test cases for comprehensive testing
- Calculation:
  - Number of decisions + 1

Cyclomatic Complexity Examples

```
public static String evenOdd(int num) {
      if (num == 0) {
            return "Even";
      } else if (num == 2) {
            return "Even";
      } else if (num == 4) {
            return "Even";
      } else if (num == 6) {
            return "Even";
      } else if (num == 8) {
            return "Even";
      } else {
            return "Odd";
      }
}
```

6



Cyclomatic Complexity Examples…



```
public stat  ring e  d(int n
      if (num == 0 || num == 2 || num == 4 || num == 6 || num == 8) {
            return "Even";
      }
      return "Odd";
}
```

6



```
public static S  venOdd(int num) {
      if (num % 2 == 0) {
            return "Even";
      }
      return "Odd";
}
```

2

Halstead's Metrics

- Metrics of the software should reflect the programming language.
- Calculations:

$\eta = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$
Length: $N = N_1 + N_2$
Volume: $V = N * \log_2 \eta$
Difficulty: $D = \eta_1 / 2 * N_2 / \eta_2$
Effort: $E = D * V$

η_1 = the number of distinct operators
η_2 = the number of distinct operands
N_1 = the total number of operators
N_2 = the total number of operands

```
main()
{
  int a, b, c, avg;
  scanf("%d %d %d", &a, &b, &c);
  avg = (a+b+c)/3;
  printf("avg = %d", avg);
}
```

Potential Final Exam Question: For the same program written in C and Python, explain why the cyclomatic complexity of the code will most likely be the same but Halstead's metric will be different?

- The C programming language has more operators/operands, conditions and branches will remain the same.

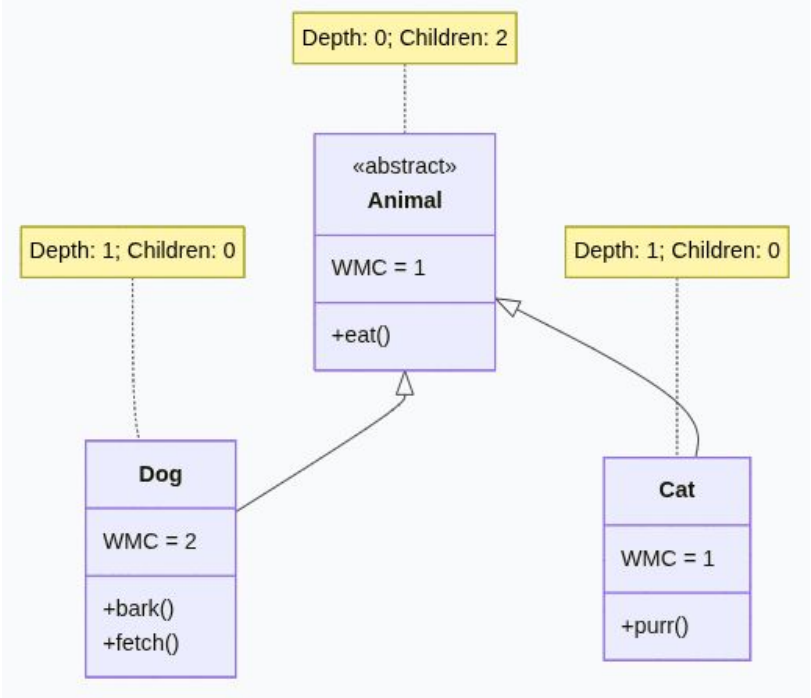
The distinct *operators* (what action to perform) are:
`main, (), {}, int, scanf, &, =, +, /, printf, ,, ;`
(n_1 = 12)

The distinct *operands* (what to apply actions to) are:
`a, b, c, avg, "%d %d %d", 3, "avg = %d"`
(n_2 = 7)

Object-Oriented Metrics

- **Weighted Methods per Class:** Weighted sum of complexities of methods defined in a class.
- **Inheritance Tree Depth:** Maximum length from node to a root in class hierarchy tree.
- **Number of Children:** Number of direct descendants (subclasses) for each class.

Object-Oriented Metrics (Example)



Depth/Children: Design Tradeoffs

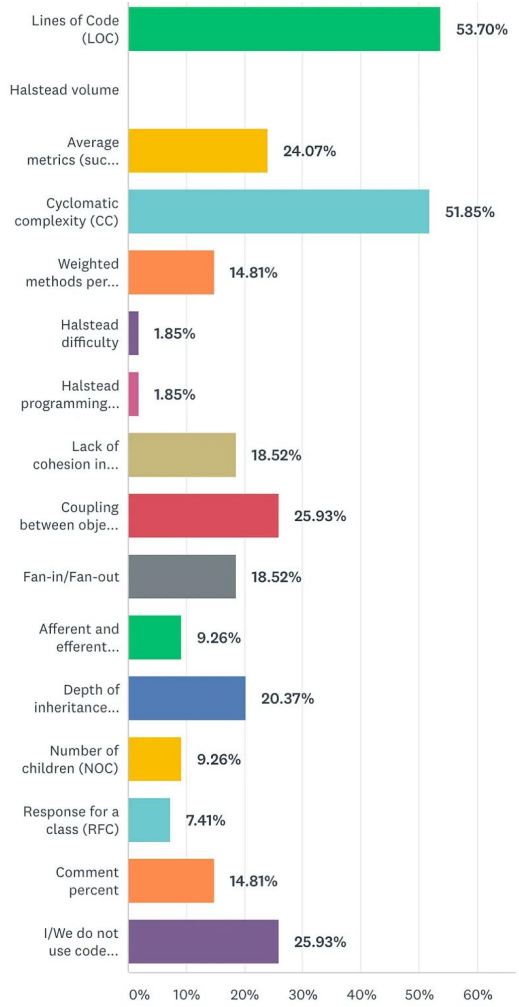
- Objects with a large number of children (depth and direct) are:
  - Difficult to modify
  - Require more testing
  - More complex and error-prone
  - **BUT,** have greater reuse of defined methods and make overall program easier to modify!

Which Metric?

- Lots of code metrics have been proposed

"The impossible holy grail" **[Fenton]**

- ***But nothing works better (predicting effort and errors) than counting lines of code (LOC)!***
  - El Emam: "Confounding Effects of Class Size on the Validity of Object-Oriented Metrics" [2001]
  - Herraiz: "Beyond Lines of Code: Do We Need More Complexity Metrics?" [2010]



And a survey of 78 software engineers found…

[Sharma]

Code Analysis

- Manual analysis of code is very valuable, but also inefficient.
- Many automated tools can programmatically analyze large software systems.
  - Static Analysis
  - Dynamic Analysis

Static Analysis

The process of automatically examining source code *without executing the application*.

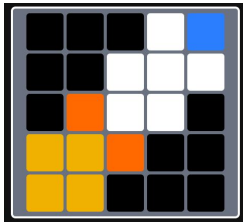
- analysis tools, code metrics, linters, code clone detection, etc.

How to do static analysis?

Convert code into a program representation

- Text matching (typically ineffective for code)
- Abstract Syntax Tree (AST)*
- Call Graphs
- Control Flow Graphs (CFG)
- Program Dependency Graph
- Tokenization*

Abstract Syntax Trees



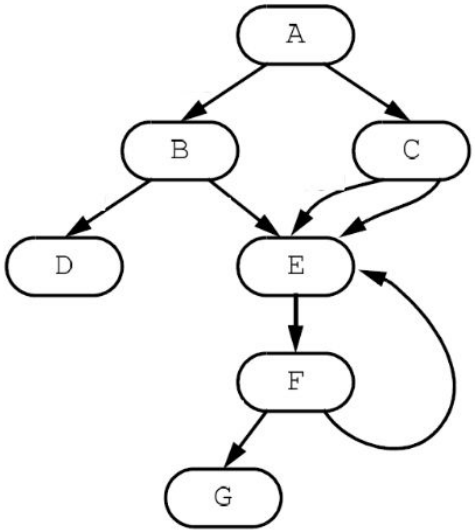
- Tree representation of source code structure
  - Node: construct (i.e. statement or loop)
  - Edge: construct relationship

```
1  r = range(5)
2  c = "white"
3  for x in r:
4    for y in r:
5      if x + y == 4 or x + y == 3 or x + y == 5:
6        if x < 2 and y > 2:
7          c = "yellow"
8        elif (x == 1 and y == 2) or (x == 2 and y == 3):
9          c = "orange"
10       elif x == 4 and y == 0:
11         c = "blue"
12       else:
13         c = "white"
14       pydle(x, y, "", c)
```

```
Module
├── Assign (r = range(5))
│   ├── target: Name(id='r')
│   └── value: Call
│       ├── func: Name(id='range')
│       └── args: Constant(5)
├── Assign (c = "white")
│   ├── target: Name(id='c')
│   └── value: Constant(value='white')
├── For (for x in r:)
│   ├── target: Name(id='x')
│   ├── iter: Name(id='r')
│   └── body:
│       └── For (for y in r:)
│           ├── target: Name(id='y')
│           ├── iter: Name(id='r')
│           └── body:
│               └── If (outer condition)
│                   ├── test: BoolOp(Or)
│                   │   ├── x + y == 4
│                   │   ├── x + y == 3
│                   │   └── x + y == 5
│                   └── body:
│                       └── If (nested if-elif-else chain)
│                           ├── if: x < 2 and y > 2
│                           │   └── Assign(c = 'yellow')
│                           └── elif: (BoolOp Or)
│                               ├── (x == 1 and y == 2)
│                               ├── (x == 2 and y == 3)
│                               │   └── Assign(c = 'orange')
│                               └── elif: x == 4 and y == 0
│                                   ├── body: Assign(c = 'blue')
│                                   └── else: Assign(c = 'white')
└── Expr (function call)
    └── Call: pydle(x, y, "", c)
        ├── func: Name(id='pydle')
        └── args: [Name(x), Name(y), Constant(""), Name(c)]
```

Call Graphs

- A directed graph representing caller-callee relationship between methods/functions
  - Node: methods/functions
  - Edges: calls



Tokenization

```
int main(){
    int i = 0;
    static int j=5;
    while(i<20){
        i=i+j;
    }
    std::cout<<"Hello World"<<i<<std::endl;
    return 0;
}
```

Remove white spaces

Tokenization (cont.)

```
int main(){
int i = 0;
static int j=5;
while(i<20){
i=i+j;
}
std::cout<<"Hello World"<<i<<std::endl;
return 0;
}
```



Tokenization (cont.)

```
int main (){
int i = 0;
int j = 5;
while (i < 20){
i  = i + j;
}
cout << "Hello World" << i << endl;
return 0;
}
```

Tokenize literals, and identifiers of types, methods, and variables.

Tokenization

```
$p $p(){
$p $p = $p;
$p $p = $p;
while($p < $p ){
$p = $p + $p;
}
$p << $p << $p << $p;
return $p;
}
```

Dynamic Analysis

The investigation of the properties of a *running* software system.

- Loggers, debuggers, etc.

How to do dynamic analysis?

Instrumentation: Modify code or runtime to monitor specific components in a system and collect data

- Instrumentation approaches:
  - Source code modification*
  - Byte code modification
  - VM modification

Source Code Instrumentation

```
public class Test {
      public static void main(String[] args) {
            if (args.length == 0) return;
            if (args.length % 2 == 0) printEven();
            else printOdd();
      }
      public static void printEven() {System.out.println(“Even”);}
      public static void printOdd() {System.out.println(“Odd”);}
}
```

Source Code Instrumentation (cont.)

- Call site instrumentation: Insert *before* each method call
- Method entry instrumentation: Insert *at entry* of each method

```
public class Test {
	public static void main(String[] args) {
		if (args.length == 0) return;
		if (args.length % 2 == 0) {
			System.out.println(“printEven() is called”);
			printEven();
		} else {
			System.out.println(“printOdd() is called”);
			printOdd();
		}
	}
	public static void printEven() {System.out.println(“Even”);}
	public static void printOdd() {System.out.println(“Odd”);}
}
```

```
public class Test {
	public static void main(String[] args) {
		if (args.length == 0) return;
		if (args.length % 2 == 0) printEven();
		else printOdd();
	}
	public static void printEven() {
		System.out.println(“printEven() is called”);
		System.out.println(“Even”);
	}
	public static void printOdd() {
		System.out.println(“printOdd() is called”);
		System.out.println(“Odd”);
	}
}
```

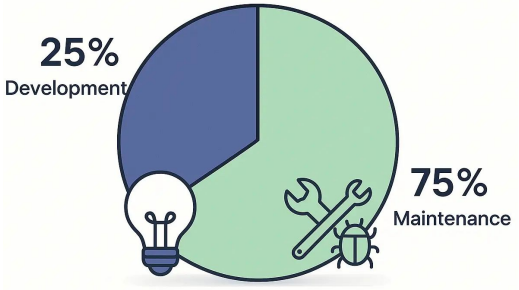
Software Maintenance

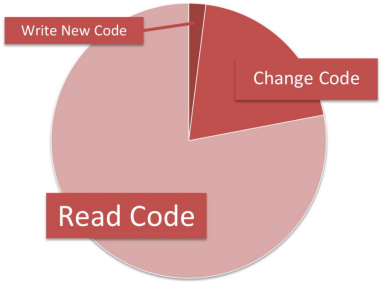
Maintenance: the process of changing, modifying, and updating software to keep up with customer needs.

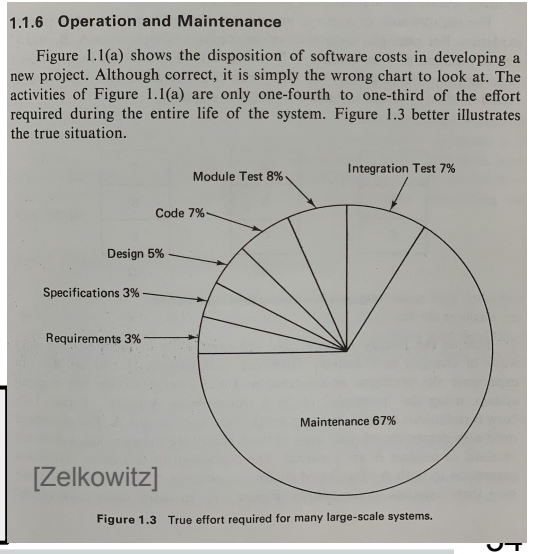
- The goal of software maintenance is to keep the software system working correctly, efficiently, and securely, and to ensure that it continues to meet the needs of the users.

Review: What is SE?

- The majority of software engineering is *not* writing code.
  - The dominant activities are **comprehension** and **maintenance**.







Maintenance in Practice

- IBM: 50-75% of costs
- Standish: 3-4x original cost
- Gartner: 55-80% of budget

[Jurkėnas]

"Understanding code is by far the activity at which professional developers spend most of their time."
– Peter Hallam, Microsoft [2006]

Case Studies

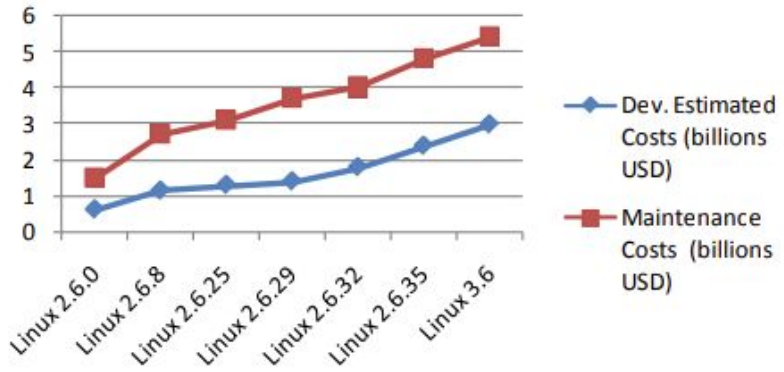


Figure 5. Evolution of Debian Linux operating system and estimated development and maintenance costs.

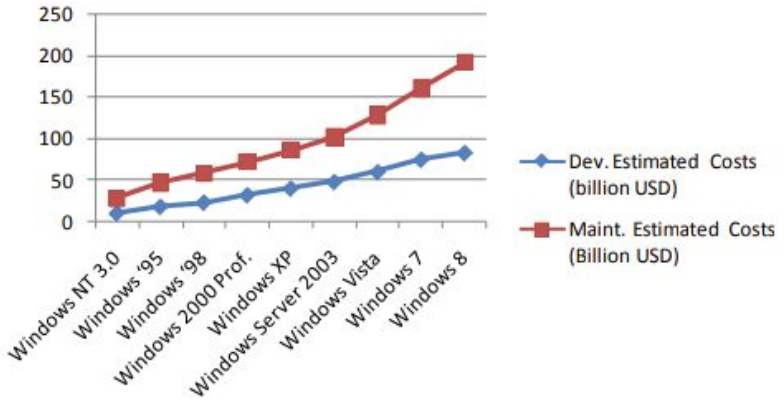


Figure 3. Evolution of Windows operating system and estimated development and maintenance costs.

Types of Software Maintenance

1. Corrective
2. Preventative
3. Perfective
4. Adaptive

Corrective Software Maintenance

- Fixing something that goes wrong in a piece of software.
  - Ex.) Finding faults and errors

** This is the most common form of software maintenance!*

Preventative Software Maintenance

- Predicting changes to keep software working for as long as possible.
  - Ex.) Upgrades, bug prevention, security updates, latent faults

Perfective Software Maintenance

- Improving software to adjust to user needs and stay relevant on the market.
  - Ex.) Adding new features, removing less effective features

Adaptive Software Maintenance

- Changing non-functional aspects of the system related to your software.
  - Ex.) Compatibility on new operating systems, cloud storage, hardware, policies and rules for usage

Common Maintenance Activities

1. Refactoring
2. Code Reviews

Refactoring

The process of modifying code for improvement *without* changing the underlying functionality and behavior of the program.

- Refactoring is not just modifying code
- There are many different types of refactoring

Change Function Declaration
Add Parameter • Change Signature • Remove Parameter • Rename Function • Rename Method

Change Reference to Value

Change Value to Reference

Collapse Hierarchy

Combine Functions into Class

Combine Functions into Transform

Consolidate Conditional Expression

Decompose Conditional

Encapsulate Collection

Encapsulate Record
Replace Record with Data Class

Encapsulate Variable
Encapsulate Field • Self-Encapsulate Field

Extract Class

Extract Function
Extract Method

Remove Dead Code

Remove Flag Argument
Replace Parameter with Explicit Methods

Remove Middle Man

Remove Setting Method

Remove Subclass
Replace Subclass with Fields

Rename Field

Rename Variable

Replace Command with Function

Replace Conditional with Polymorphism

Replace Constructor with Factory Function
Replace Constructor with Factory Method

Replace Control Flag with Break
Remove Control Flag

Replace Derived Variable with Query

Replace Error Code with Exception

Extract Superclass

Extract Variable
Introduce Explaining Variable

Hide Delegate

Inline Class

Inline Function
Inline Method

Inline Variable
Inline Temp

Introduce Assertion

Introduce Parameter Object

Introduce Special Case
Introduce Null Object

Move Field

Move Function
Move Method

Move Statements into Function

Move Statements to Callers

Parameterize Function
Parameterize Method

Preserve Whole Object

Pull Up Constructor Body

Pull Up Field

Pull Up Method

Push Down Field

Replace Exception with Precheck
Replace Exception with Test

Replace Function with Command
Replace Method with Method Object

Replace Inline Code with Function Call

Replace Loop with Pipeline

Replace Magic Literal
Replace Magic Number with Symbolic Constant

Replace Nested Conditional with Guard Clauses

Replace Parameter with Query
Replace Parameter with Method

Replace Primitive with Object
Replace Data Value with Object • Replace Type Code with Class

Replace Query with Parameter

Replace Subclass with Delegate

Replace Superclass with Delegate
Replace Inheritance with Delegation

Replace Temp with Query

Replace Type Code with Subclasses
Extract Subclass • Replace Type Code with State/Strategy

Return Modified Value

Separate Query from Modifier

Slide Statements
Consolidate Duplicate Conditional Fragments

Split Loop

Split Phase

Split Variable
Remove Assignments to Parameters • Split Temp

Substitute Algorithm

72 refactorings introduced ed by [Fowler et al.]

Refactoring Example

```
class Gorilla {
  int paws() {
     return 4;
  }
}
```

INTRODUCE EXPLAINING VARIABLE

```
class Gorilla{
  int paws(){
     int pawCount = 4;
     return pawCount;
  }
}
```

EXTRACT INTERFACE

```
class Gorilla implements Primate {
  int paws() {
    int pawCount = 4;
    return pawCount;
  }
}
...
interface Primate {
  abstract int paws();
}
```

RENAME METHOD

```
class Gorilla implements Primate{
  int feet() {
    int pawCount = 4;
    return pawCount;
  }
}
...
interface Primate {
  abstract int feet();
}
```

Why Refactor?

- The idea behind refactoring is to acknowledge that *it will be difficult to get a design right the first time!*
- As a program's requirements change, the design may also need to change.
- Refactoring provides techniques for evolving the design in small incremental steps.

Benefits

- Often code size is reduced after refactoring
- Confusing structures are transformed into simpler structures
  - ***Which are easier to maintain and understand***

Floss vs. Root Canal Refactoring

Describes when refactoring is done and why. 🦷

Root Canal: Large changes to software code.

- Painful, expensive, the result of long periods of neglect
- Not considered best practice ✘

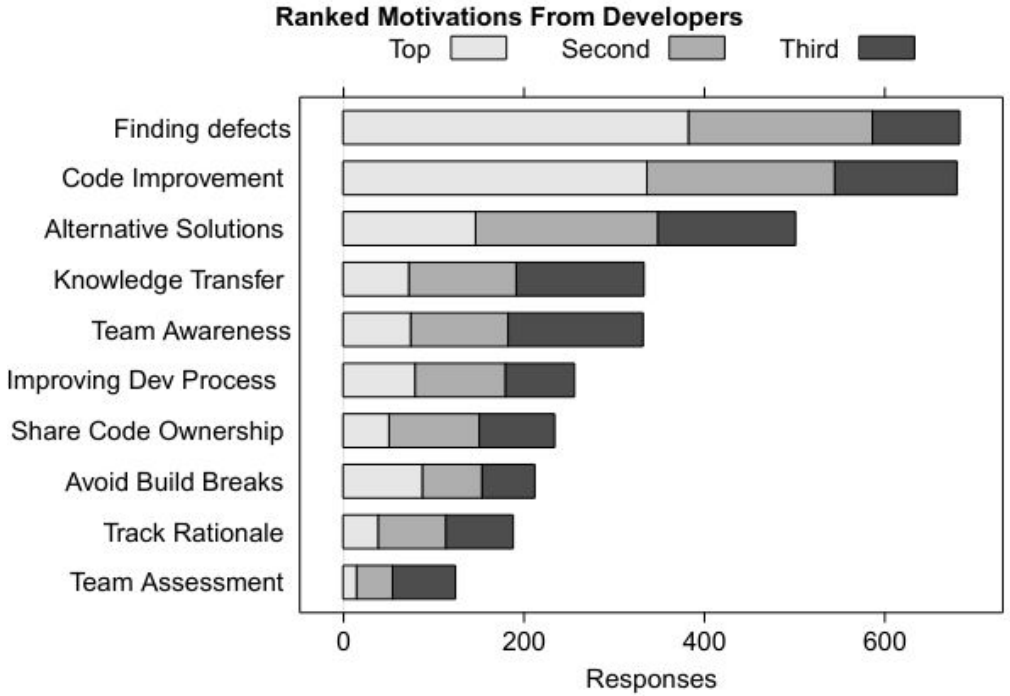
Floss: Smaller and more frequent code changes software engineers know they should do.

- Achieves an immediate goal
- As often as “every few minutes”
- Considered best practice ✅

Code Reviews

- The process of manually inspecting source code changes.
  - Human code analysis (also known as peer code reviews)
  - Usually performed by a developer other than the author
- Goal: Inspect code before integration to improve software quality.

Why Code Reviews?



[Bacchelli]

Code Reviews at Google

- "All code that gets submitted needs to be reviewed by at least one other person, and either the code writer or the reviewer needs to have readability in that language. Most people use Mondrian to do code reviews, and obviously, we spend a good chunk of our time reviewing code."

-- Amanda Camp, Software Engineer, Google

Code reviews at Yelp

- “At Yelp we use review-board. An engineer works on a branch and commits the code to their own branch. The reviewer then goes through the diff, adds inline comments on review board and sends them back. The reviews are meant to be a dialogue, so typically comment threads result from the feedback. Once the reviewer's questions and concerns are all addressed they'll click "Ship It!" and the author will merge it with the main branch for deployment the same day.”

-- Alan Fineberg, Software Engineer, Yelp

Code reviews at WotC

- “At Wizards we use Perforce for SCM. I work with stuff that manages rules and content, so we try to commit changes at the granularity of one bug at a time or one card at a time. Our team is small enough that you can designate one other person on team as a code reviewer. Usually you look at code sometime that week, but it depends on priority. It’s impossible to write sufficient test harnesses for the bulk of our game code, so code reviews are absolutely critical.”

-- Jake Englund, Software Engineer, MtGO

Code reviews at Facebook

- "At Facebook, we have an internally-developed web-based tool to aid the code review process. Once an engineer has prepared a change, she submits it to this tool, which will notify the person or people she has asked to review the change, along with others that may be interested in the change -- such as people who have worked on a function that got changed.

  At this point, the reviewers can make comments, ask questions, request changes, or accept the changes. If changes are requested, the submitter must submit a new version of the change to be reviewed. All versions submitted are retained, so reviewers can compare the change to the original, or just changes from the last version they reviewed. Once a change has been submitted, the engineer can merge her change into the main source tree for deployment to the site during the next weekly push, or earlier if the change warrants quicker release."

  - Ryan McElroy, Software Engineer, Facebook

Project 2 Install-Fest

Goal: Team set up for Project 2.

Activities:

https://github.com/CS3704-VT/Course/tree/main/Projects/Project2/install-fest

1. Fork Project 2 repository (one team member)
2. Clone forked repository
3. Agent-template and dependency setup
   - Baker
   - Manual
4. Run `/orient` and `/learning-goal` skills
5. Submit orientation and team learning goals on Canvas

Next Class

10/15: Guest Lecture

- Matt McHugh (Excella)

10/20: AI Coding Overview and Workshop

Announcements

- HW2 due tomorrow (10/14) before midnight
- PM2.1 due 10/23 before midnight

References

- RS Pressman. *“Software engineering: a practitioner's approach”.*
- Tushar Sharma, et al. “*Do We Need Improved Code Quality Metrics?*”, 2020
- Rachel Potvin, et al. *“Why Google Stores Billions of Lines of Code in a Single Repository”*. 2016
- Welker, Kurt D. "The software maintainability index revisited." CrossTalk 14 (2001): 18-21.
- Vukelja, Ljiljana, Lothar Müller, and Klaus Opwis. "Are engineers condemned to design? A survey on software engineering and UI design in Switzerland." *Human-Computer Interaction–INTERACT 2007: 11th IFIP TC 13 International Conference, Rio de Janeiro, Brazil, September 10-14, 2007, Proceedings, Part II 11*. Springer Berlin Heidelberg, 2007.
- Zhang, Cheng, and David Budgen. "What do we know about the effectiveness of software design patterns?." *IEEE Transactions on Software Engineering* 38.5 (2011): 1213-1231.
- Na Meng and Barbara Ryder
- Chris Parnin
- Sarah Heckman